Uniwersytet Łódzki
Wydział Matematyki i Informatyki

Temat ćwiczenia:

Grupowanie danych

Sprawozdanie z przedmiotu: ***Zaawansowane Metody Eksploracji***
Studia stacjonarne 2-go stopnia
Kierunek: *Analiza Danych, sem. II*
Prowadzący: *prof. dr hab. inż. Grzegorz Dudek*

Data wykonania ćwiczenia:	26.03.2025
Data złożenia sprawozdania:	09.04.2025
Data zaliczenia:	

Wykonali:	Ocena:
Dawid Sporysiak	

Nr sekcji: **3.19**

A. Cel ćwiczenia

Głównym celem przeprowadzonych analiz było zastosowanie metod klasteryzacji do identyfikacji naturalnych grup genów o podobnych profilach ekspresji w czasie. Wykorzystano różne techniki grupowania (K-średnich, hierarchiczne z complete linkage) oraz redukcję wymiarowości (PCA) oraz ich ilustracje na wykresach.

Teoria do sprawozdania

Analiza PCA działa przez przekształcenie skorelowanych zmiennych w zestaw nieskorelowanych składowych głównych, ułożonych w kolejności malejącej wariancji. Metoda ta znajduje nowe osie w przestrzeni danych - pierwsza składowa (PC1) o maksymalnej wariancji, druga (PC2) - prostopadle do pierwszej, obejmując kolejne największe zróżnicowanie, i tak dalej. PCA standaryzuje dane, oblicza macierz kowariancji, wyznacza wektory własne (kierunki składowych) i wartości własne (wielkość wariancji), pozwalając na redukcję wymiarowości przy minimalnej utracie informacji.

Metoda K-średnich to algorytm grupowania, który dąży do minimalizacji sumy kwadratów odległości punktów od centroidów ich klastrów. Działa w trzech etapach: losowa inicjalizacja centroidów, przypisanie punktów do najbliższego centroidu metodą euklidesową, aktualizacja pozycji centroidów jako średnich arytmetycznych przypisanych punktów. Proces powtarza się aż do stabilizacji centroidów, przy czym wynik zależy od losowej inicjalizacji, stąd praktyka wielokrotnego uruchamiania algorytmu.

Dendrogram w grupowaniu hierarchicznym wizualizuje proces aglomeracyjnego łączenia obiektów w hierarchiczną strukturę klastrów. Metoda zaczyna od traktowania każdego punktu jako osobny klaster, następnie iteracyjnie łączy najbliższe pary klastrów według wybranej miary podobieństwa (np. odległość najdalszych sąsiadów w metodzie 'complete linkage'). Wynik przedstawiany jest jako drzewo, gdzie długość gałęzi odzwierciedla odległość między łączonymi klastrami, pozwalając na określenie naturalnego podziału danych poprzez analizę najdłuższych pionowych linii dendrogramu.

B. Treść zadania

Zadanie 1

Treść:

1. Zaimportuj niezbędne moduły (uwaga – należy zainstalować pakiet ISLP: pip install ISLP)

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
from sklearn.cluster import (KMeans , AgglomerativeClustering)
from sklearn.preprocessing import StandardScaler
from sklearn.decomposition import PCA
from sklearn.metrics import silhouette_score
from scipy.cluster.hierarchy import (dendrogram , cut_tree)
from ISLP.cluster import compute_linkage
from statsmodels.datasets import get_rdataset
```

Rozwiązanie zadania:

```
#zadanie 1
#należało pobrać kilka bibliotek do poprawnego działania kodu, między innymi ISLP oraz te które wcześniej nie
były podstawowo zainstalowane w google collab
#!pip install ISLP – instalacja ISLP
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
#biblioteka do rysowania wykresów
from sklearn.cluster import (KMeans , AgglomerativeClustering)
#biblioteka do kśrednich i klasteryzacji
from sklearn.preprocessing import StandardScaler
from sklearn.decomposition import PCA
#biblioteka do wykorzystania metody PCA
from sklearn.metrics import silhouette_score
#biblioteka do rysowania sylwetki
from scipy.cluster.hierarchy import (dendrogram , cut_tree)
#biblioteka do rysowania dendogramu wraz z odcięciem
from ISLP.cluster import compute_linkage
from statsmodels.datasets import get_rdataset
from ISLP import load_data
```

Wniosek:

W ramach zadania zaimportowano niezbędne biblioteki do analizy danych i klasteryzacji, takie jak pandas, numpy, matplotlib oraz funkcje z sklearn i scipy. Ponieważ niektóre pakiety (np. ISLP) nie są domyślnie dostępne w Google Colab, konieczna była ich dodatkowa instalacja.

Zadanie 2

Treść:

2. Wczytaj i zmodyfikuj zbiór danych:

```
data= pd.read_csv("../content/drive/MyDrive/Colab
Notebooks/GD_dane_genetyczne.csv")
geny = data.iloc[:,0].tolist()
ekspresja = data.iloc[:,1:].to_numpy()
czas = data.columns[1:].to_numpy()
czas = list(map(float, czas))

#modyfikacja danych
nr_gr = ?
r_k = ?
np.random.seed(int(nr_gr*r_k))
ekspresja = ekspresja + np.random.rand(614, 7) * 0.1 - 0.05
```

gdzie za `nr_gr` wstaw numer swojej sekcji a za `r_k` aktualny rok kalendarzowy.

Podejrzyj zmienne.

Rozwiązanie zadania:

```
#Zczytanie danych do sprawozdania
data = pd.read_csv("GD_dane_genetyczne.csv")
#Fragment danych bez dodania szumu
print(data.head(5))
#Wybranie potrzebnych danych
geny = data.iloc[:, 0].tolist()
ekspresja = data.iloc[:, 1:].to_numpy()
czas = data.columns[1:].to_numpy()
czas = list(map(float, czas))
#wartości do ustawienia ziarna losowego
nr_gr = 3.19
r_k = 2025
np.random.seed(int(nr_gr * r_k))
#ekspresja oraz szum
noise = np.random.rand(ekspresja.shape[0], ekspresja.shape[1]) * 0.1 - 0.05
ekspresja = ekspresja + noise
data.iloc[:, 1:] = ekspresja
#fragment danych po dodaniu szumu
print(data.head(5))

#Skalowanie danych
scaler = StandardScaler()
ekspresja_scaled = scaler.fit_transform(ekspresja)

#Analiza PCA
pca = PCA()
scores = pca.fit_transform(ekspresja_scaled)
```

Wynik:

```
         -      0    9.5   11.5   13.5   15.5   18.5   20.5
0   SS DNA -0.131  1.699 -0.026  0.365 -0.246  0.478  0.435
1  YAL003W  0.305  0.146 -0.129 -0.444 -0.707 -1.499 -1.935
2  YAL012W  0.157  0.175  0.467 -0.379 -0.520 -1.279 -2.125
3  YAL026C  0.246  0.796  0.384  0.981  1.020  1.646  1.157
4  YAL034C -0.235  0.487 -0.184 -0.669 -1.006  2.369  2.611
         -         0       9.5      11.5      13.5      15.5      18.5
0   SS DNA -0.094397  1.709554 -0.072555  0.396467 -0.203250  0.521312
1  YAL003W  0.348610  0.146091 -0.111695 -0.411606 -0.665159 -1.535658
2  YAL012W  0.137279  0.174058  0.462039 -0.380087 -0.547188 -1.242051
3  YAL026C  0.215047  0.773771  0.339523  0.934130  1.066630  1.631395
4  YAL034C -0.186892  0.460883 -0.160262 -0.712738 -0.958103  2.400846

       20.5
0  0.437996
1 -1.961457
2 -2.168959
3  1.107540
4  2.635375
```

Wniosek:

W tej części zadania wczytano dane genetyczne z pliku CSV i przygotowano je do dalszej analizy. Najpierw wyodrębniono geny i ich ekspresję w czasie, a następnie dodano sztuczny szum do danych ekspresji genów, używając stałej ziarna losowego opartej na numerze 3.19 i roku 2025. Dane następnie przeskalowano i przeprowadzono wstępną analizę PCA, która pozwoli na redukcję wymiarowości danych.

Zadanie 3

Treść:

Zwizualizuj dane wykorzystując metodę PCA i pokaż wykresy wariancji wyjaśnianej przez poszczególne składowe PCA (wykorzystaj kod z komórek [50]-[52] rozdz. 12.5.4)

Rozwiązanie zadania:

```
#Układ danych, wiersze i kolumny
ekspresja.shape
print(ekspresja)
#Dane
scaler = StandardScaler()
X_scaled = scaler.fit_transform(ekspresja)
#PCA
pca = PCA()
X_pca = pca.fit_transform(X_scaled)
#Wykres skumulowanej wyjaśnianej wariancji
```

```
plt.figure(figsize=(12, 5))
plt.subplot(1, 2, 2)
plt.plot(range(1, pca.n_components_ + 1),
        (pca.explained_variance_ratio_),
        color='green')
plt.xlabel('Numer składowej głównej (PCA)')
plt.ylabel('Wariancja wyjaśniona przez pojedyncze składowe PCA')
plt.title('Przyrost wyjaśnianej wariancji przez kolejne składowe PCA')
plt.grid(alpha=0.3)
plt.tight_layout()
plt.show()
```

Wynik:

```
[[-0.09439729  1.70955352 -0.07255491 ... -0.20324981  0.52131172
   0.43799559]
 [ 0.34861003  0.14609058 -0.11169529 ... -0.66515874 -1.53565815
  -1.96145655]
 [ 0.13727902  0.17405822  0.46203859 ... -0.54718797 -1.24205108
  -2.16895856]
 ...
 [ 0.08043691  0.51856936  1.60686781 ...  1.61694662  0.95708965
   2.08299332]
 [-0.24188157  0.18839239 -0.34805563 ...  0.88108197  1.63529874
   3.06989432]
 [-0.23435912  0.15716049  0.02382084 ...  1.11417554  0.83352468
   2.35577753]]
```

Wniosek:

Na poniższym wykresie przedstawiona jest wyjaśniona wariancja w zależnośći od liczby składowych. Odpowiednim wyborem jest punkt grzbietowy wykresu, czyli 3, gdzie łączna wyjaśniona wariancja jest na poziomie około 85%. Oznacza to, że 3 zmienne wyjaśniają rozrzut w 85% co jest bardzo dobrym wynikiem, do reduckji wymiarów z 7 do 3 bez utraty ważnych danych.

Zadanie 4

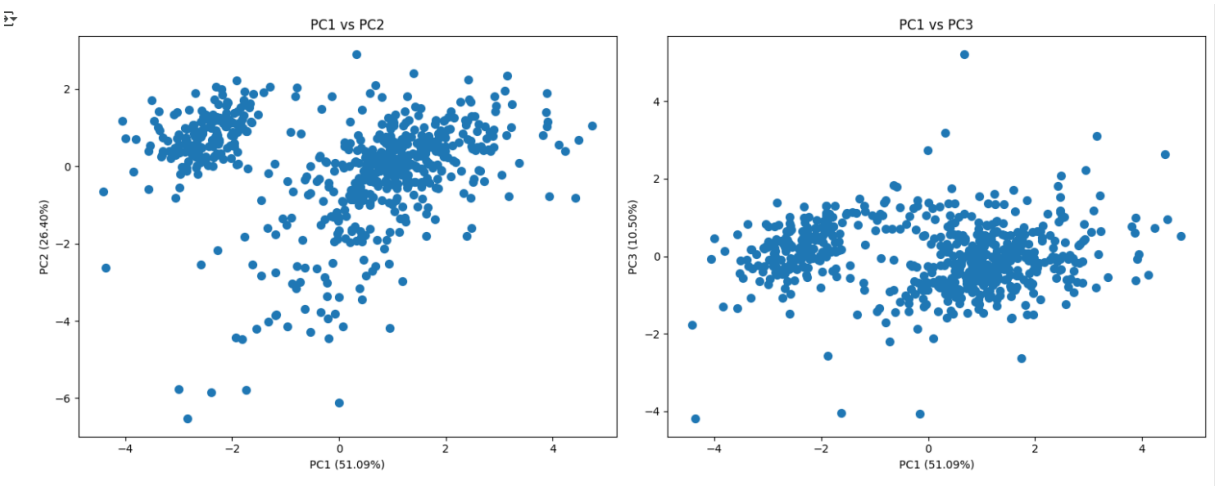
Treść:

Pogrupuj profile ekspresji genów za pomocą metody *k*-średnich na 16 grup (patrz komórka [33] rozdz. 12.5.3).

Rozwiązanie zadania:

```
fig, axes = plt.subplots(1, 2, figsize=(15,6))

# PC1 vs PC2
ax = axes[0]
scatter = ax.scatter(scores[:,0], scores[:,1], marker='o', s=50)
ax.set_xlabel('PC1 (%.2f%%)' % (pca.explained_variance_ratio_[0]*100))
ax.set_ylabel('PC2 (%.2f%%)' % (pca.explained_variance_ratio_[1]*100))
ax.set_title('PC1 vs PC2')

# PC1 vs PC3
ax = axes[1]
scatter = ax.scatter(scores[:,0], scores[:,2], marker='o', s=50)
ax.set_xlabel('PC1 (%.2f%%)' % (pca.explained_variance_ratio_[0]*100))
ax.set_ylabel('PC3 (%.2f%%)' % (pca.explained_variance_ratio_[2]*100))
ax.set_title('PC1 vs PC3')

plt.tight_layout()
plt.show()
```

Wynik:



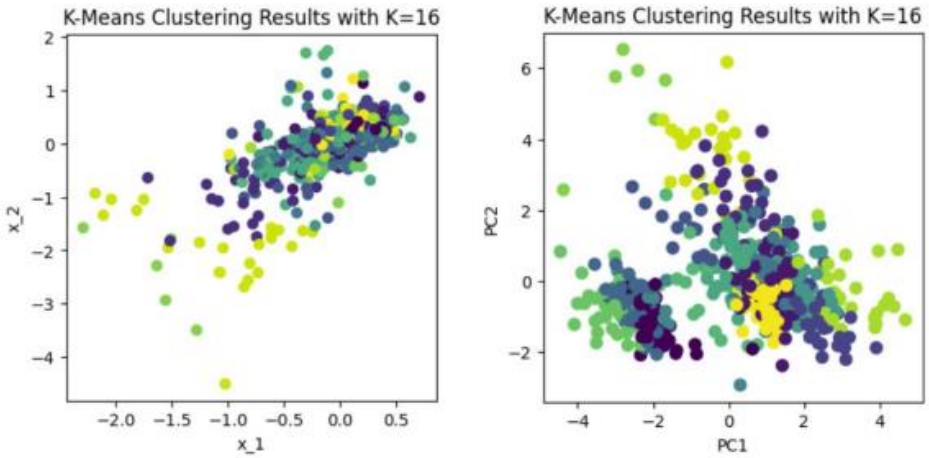
Wniosek:

W tej części zadania przedstawiono wizualizację wyników analizy PCA za pomocą wykresów rozrzutu dla dwóch par głównych składowych (PC1 vs PC2 oraz PC1 vs PC3). Z wykresów wyraźnie widać, że punkty układają się w charakterystyczne skupiska, co sugeruje możliwość istnienia naturalnych grup w danych genetycznych.

Zadanie 5

Treść:

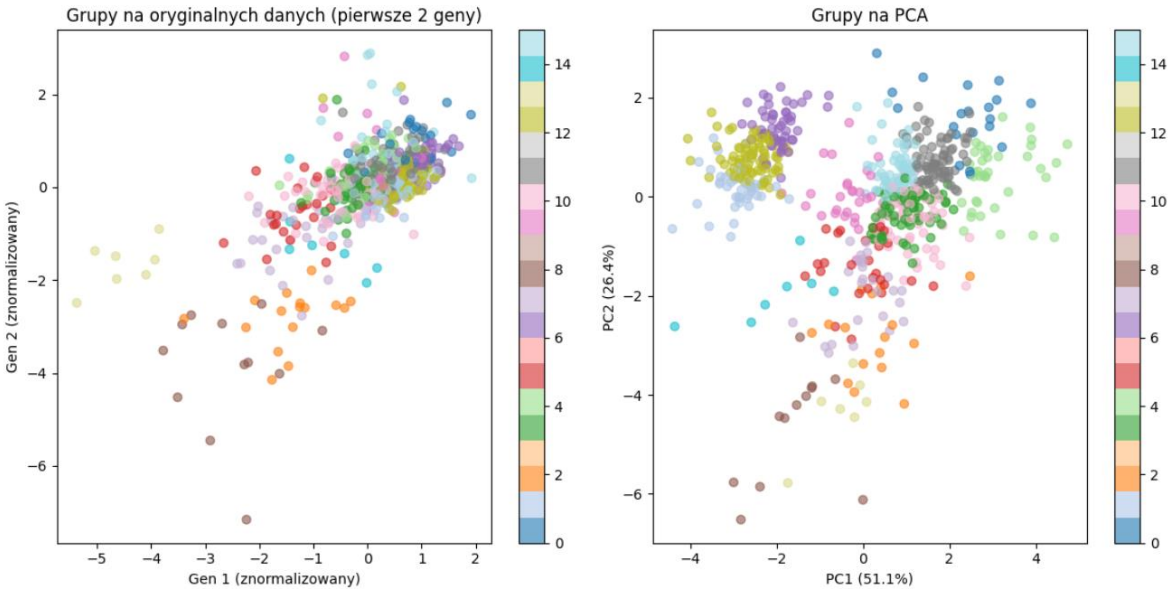
Zwizualizuj wyniki w postaci wykresów punktowych – we współrzędnych oryginalnych (wybrane atrybuty, np. x_1 i x_2) oraz we współrzędnych PCA (np. pierwsza i druga składowa główna) ([35], [50], [51]).



Rozwiązanie zadania:

```
from sklearn.cluster import KMeans
import matplotlib.pyplot as plt

k = 16
grupy = KMeans(n_clusters=k, random_state=42).fit_predict(X_pca)

fig, (ax1, ax2) = plt.subplots(1, 2, figsize=(12, 6))

# Wykres oryginalnych danych
ax1.scatter(X_scaled[:, 0], X_scaled[:, 1], c=grupy, cmap='tab20')
ax1.set_title('Grupy na oryginalnych danych')
ax1.set_xlabel('Gen 1')
ax1.set_ylabel('Gen 2')

# Wykres PCA
ax2.scatter(X_pca[:, 0], X_pca[:, 1], c=grupy, cmap='tab20')
ax2.set_title('Grupy na PCA')
ax2.set_xlabel(f'PC1 ({pca.explained_variance_ratio_[0]*100:.1f}%)')
ax2.set_ylabel(f'PC2 ({pca.explained_variance_ratio_[1]*100:.1f}%)')

plt.tight_layout()
plt.show()

print("Liczba genów w grupach:")
for i in range(k):
    print(f"Grupa {i+1}: {(grupy == i).sum()}")
```

Wynik:




```
Liczba genów w grupach:
1      20
2      42
3      16
4      68
5      38
6      26
7      52
8      26
9      12
10     28
11     58
12     71
13     79
14      7
15      7
16     64
```

Wniosek:

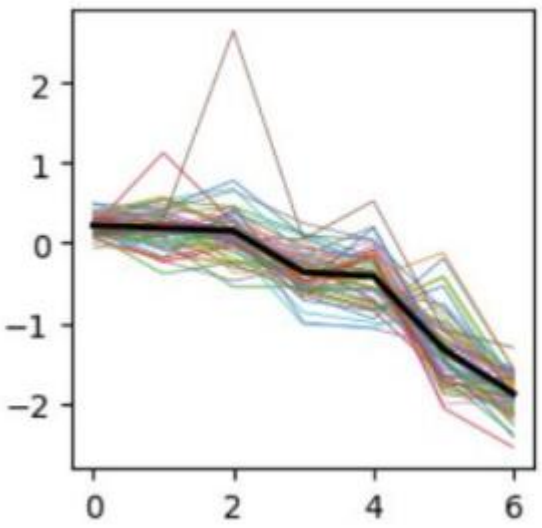
W tej części zastosowano algorytm K-means do klasteryzacji danych genetycznych, porównując wyniki w przestrzeni oryginalnych cech i po redukcji wymiarów PCA. Wybór k=16 grup ujawnił istotne różnice w strukturze danych - podczas gdy w przestrzeni oryginalnej genów skupienia były słabo widoczne, w przestrzeni PCA (gdzie PC1 i PC2 wyjaśniają łącznie ~85% wariancji) uzyskano wyraźnie lepszą separację grup. Obserwacja nierównomiernego rozkładu genów między klastrami (od pojedynczych genów w najmniejszych do kilkudziesięciu w największych grupach) sugeruje, że przyjęta wartość k=16 mogła być zbyt wysoka, prowadząc do

nadmiernego podziału naturalnych skupień. Jednocześnie wyraźne zgrupowanie punktów w przestrzeni PCA potwierdza zasadność redukcji wymiarowości przed analizą skupień.

Zadanie 6

Treść:

6. Zwizualizuj profile należące do poszczególnych grup i centroidy tych grup. 16 wykresów postaci:

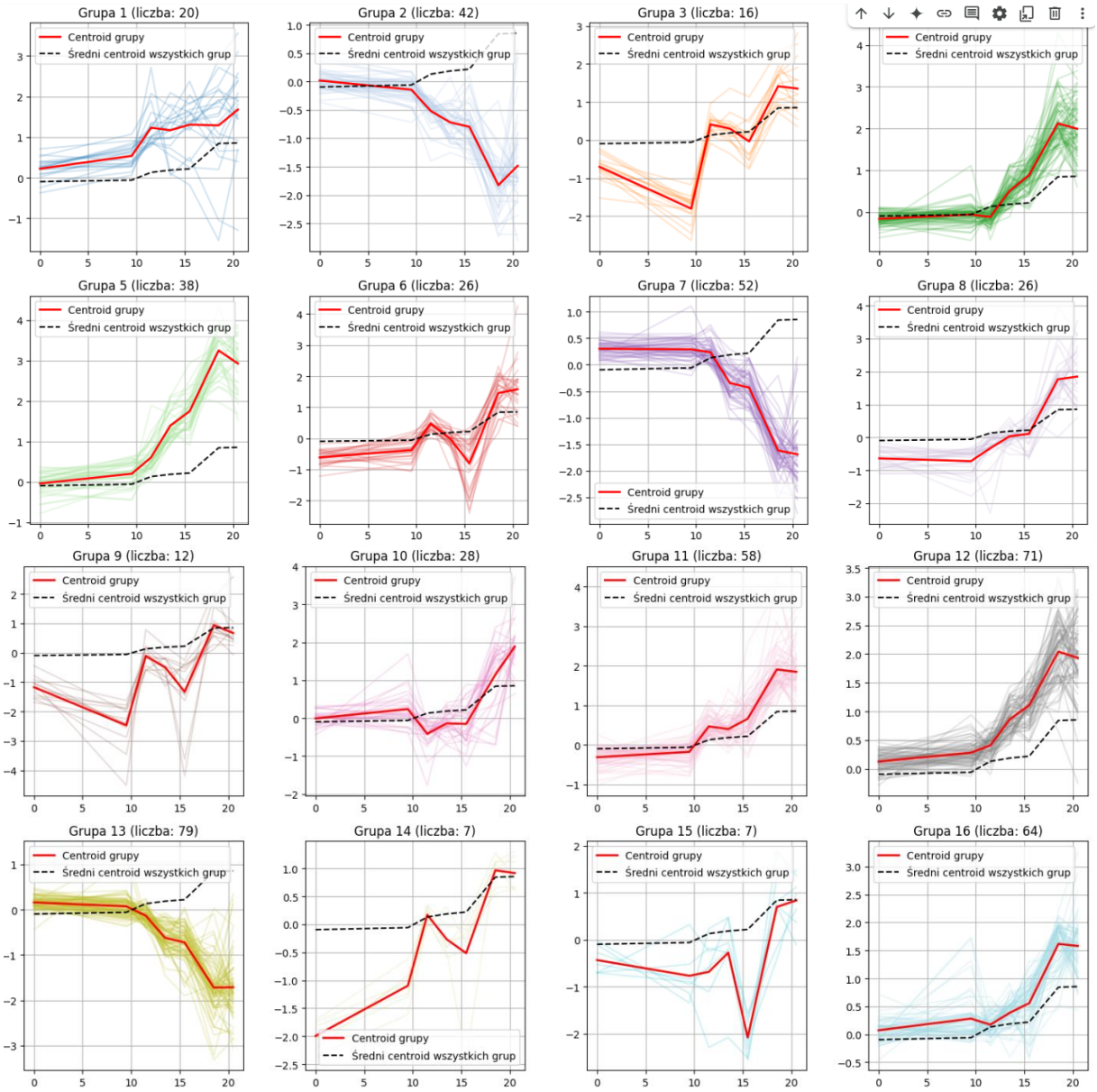


Rozwiązanie zadania:

```
import matplotlib.pyplot as plt

#Grupowanie i centroidy dla k-means
grupy = KMeans(n_clusters=16, random_state=1).fit_predict(scores)
centroidy = scaler.inverse_transform(km.cluster_centers_ @ pca.components_)
srednia_globalna = ekspresja.mean(axis=0)

plt.figure(figsize=(15, 15))

for i in range(16):
   plt.subplot(4, 4, i+1)

   #Profile genów w grupie
   for gen in ekspresja[grupy == i]:
      plt.plot(czas, gen, color=f'C{i}', alpha=0.1)

   #Centroidy lokalne i globalny
   plt.plot(czas, centroidy[i], 'r-', linewidth=2, label='Centroid')
```

```
    plt.plot(czas, srednia_globalna, 'k--', label='Średnia globalna')

    plt.title(f'Grupa {i+1} (n={sum(grupy==i)})')
    plt.grid(True)
    if i == 0: plt.legend()

plt.tight_layout()
plt.show()
```

Wynik:



Wniosek:

- W tej części przeprowadzono szczegółową analizę profili ekspresji genów w poszczególnych klastrach. Widoczne są wyraźne różnice w profilach ekspresji między klastrami, co potwierdza skuteczność grupowania
- Niektóre klastry (np. te liczne) wykazują bardziej spójne profile niż małe grupy
- Centroidy małych grup są bardziej wrażliwe na dodanie nowych genów
- Globalna średnia jest silnie zdominowana przez profile z najliczniejszych klastrów

Zadanie 7

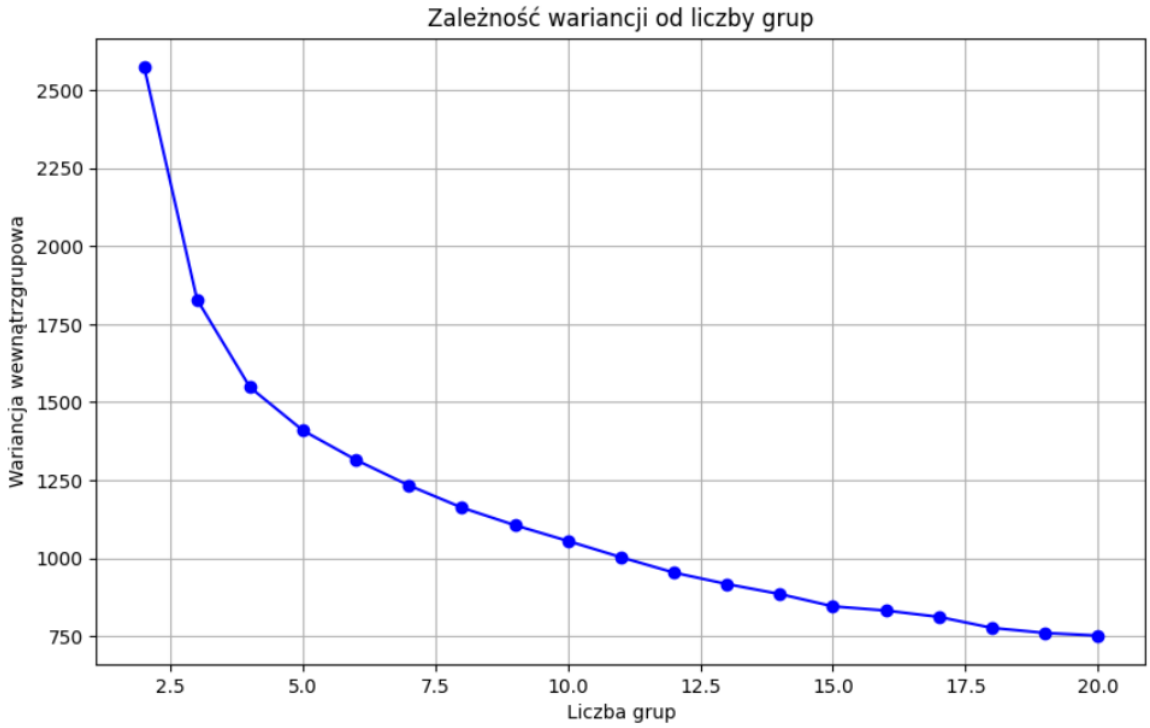
Treść:

Wykreśl zależność wariancji wewnątrzgrupową (`kmeans.inertia_`) od liczby grup (`n_clusters`).

Rozwiązanie zadania:

```
cluster_range = range(2, 21)
inertias = []

#Obliczenie wariancji zależne od ilości grup
for n in cluster_range:
    kmeans = KMeans(n_clusters=n, random_state=1, n_init=20).fit(X_scaled)
    inertias.append(kmeans.inertia_)

plt.figure(figsize=(10, 6))
plt.plot(cluster_range, inertias, 'bo-')
plt.xlabel('Liczba grup')
plt.ylabel('Wariancja wewnątrzgrupowa')
plt.title('Zależność wariancji od liczby grup')
plt.grid(True)
plt.show()
```

Wynik:



Wniosek:

Analiza metody łokcia dla różnych wartości k wykazała, że optymalna liczba klastrów dla tych danych wynosi 3 lub 4, co odpowiada punktom załamania na wykresie zależności wariancji wewnątrzgrupowej od liczby grup. Obserwujemy, że:

1. Dla k=3-4 następuje wyraźne spowolnienie spadku wariancji
2. Powyżej k=4 przyrost liczby grup daje już tylko niewielką poprawę
3. Krzywa ma charakterystyczny kształt "łokcia" w tym zakresie, uzasadnione może być skupienie się na 3-4 głównych grupach genów
4. Dalsze zwiększanie liczby klastrów prowadzi do nadmiernego podziału bez istotnej poprawy jakości grupowania

Zadanie 8

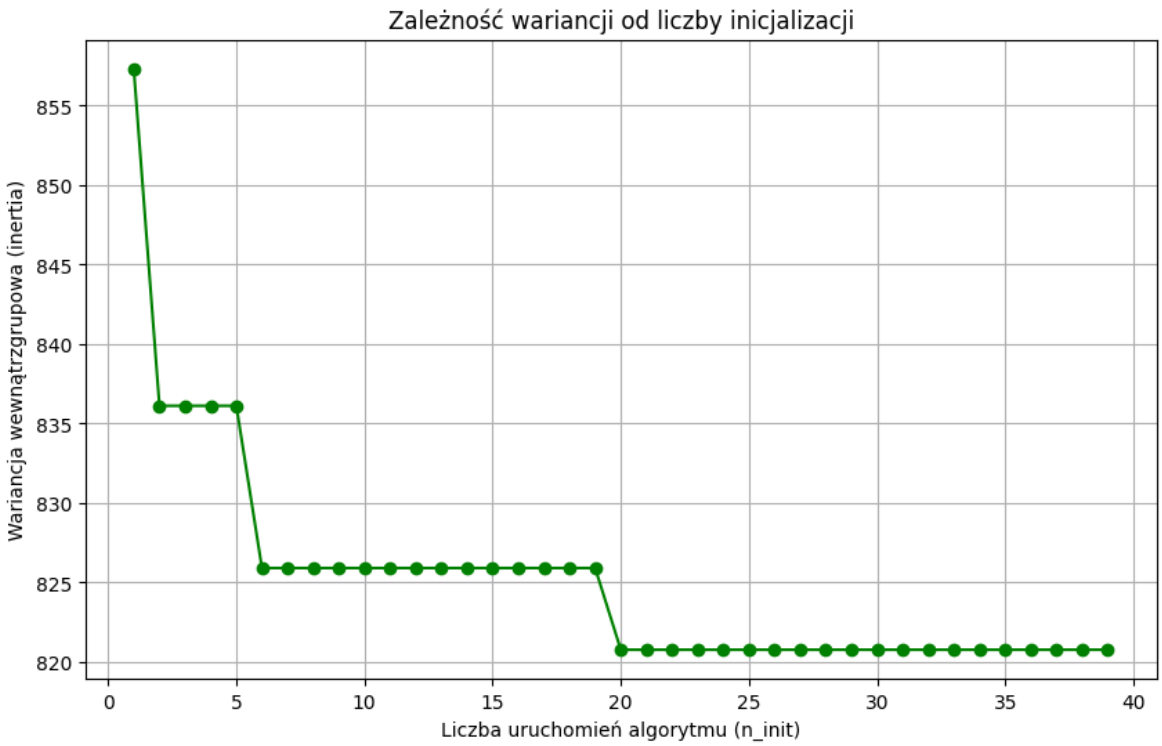
Treść:

Wykreśl zależność wariancji wewnątrzgrupową (`kmeans.inertia_`) od liczby uruchomień algorytmu (`n_init`).

Rozwiązanie zadania:

```
n_init_range = range(1, 40)
inertias_ninit = []

for n in n_init_range:
    kmeans = KMeans(n_clusters=16, random_state=1, n_init=n).fit(X_scaled)
    inertias_ninit.append(kmeans.inertia_)

plt.figure(figsize=(10, 6))
plt.plot(n_init_range, inertias_ninit, 'go-')
plt.xlabel('Liczba uruchomień algorytmu (n_init)')
plt.ylabel('Wariancja wewnątrzgrupowa (inertia)')
plt.title('Zależność wariancji od liczby inicjalizacji')
plt.grid(True)
plt.show()
```

Wynik:



Wniosek:

Wykres pokazuje, że optymalną liczbą uruchommień jest n=6 wtedy wariancja jest względnie niska w porównaniu do mniejszych liczb uruchomień, a dla wyższych n wariancja już nieznacznie się poprawia. Dalsze zwiększanie n_init przynosi minimalne korzyści. Powyżej n_init=8 krzywa się wypłaszcza, wskazując na osiągnięcie powtarzalności.

Zadanie 9

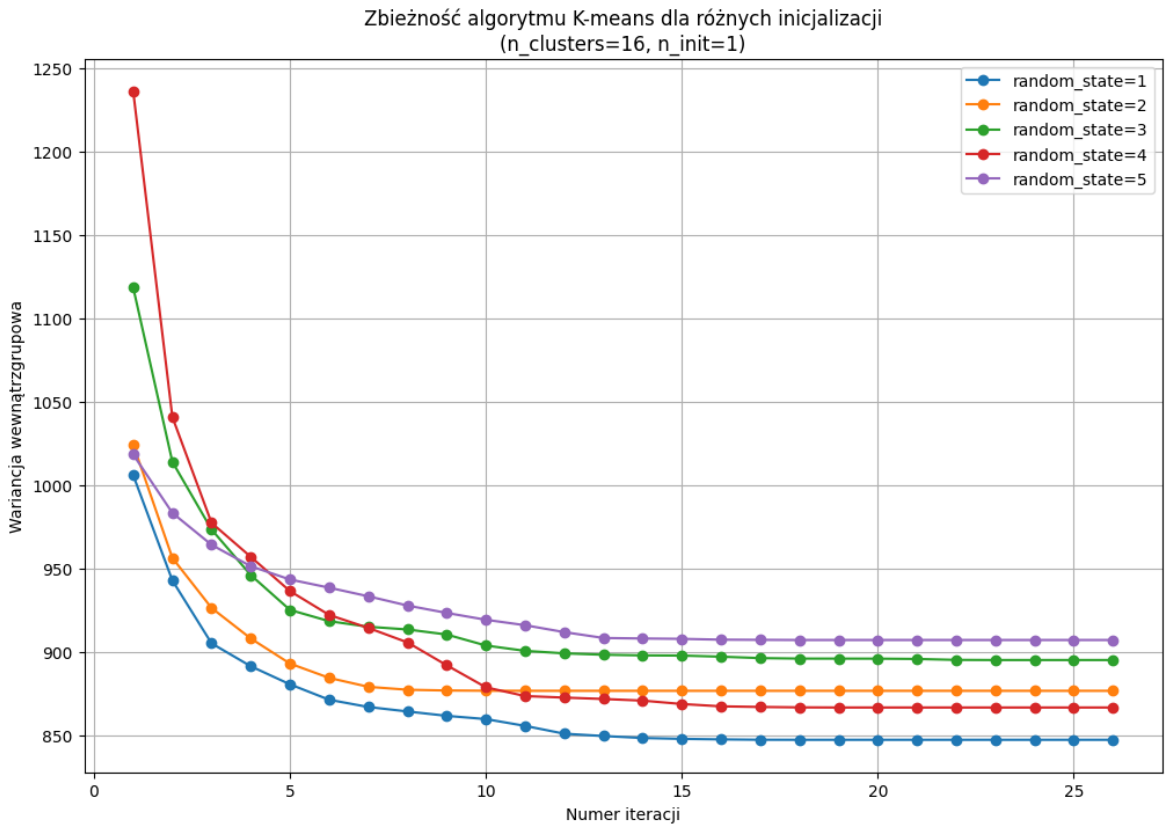
Treść:

Wykreśl wykresy obrazujące wariancję wewnątrzgrupową (`kmeans.inertia_`) w kolejnych iteracjach dla pięciu różnych stanów początkowych (`random_state`). Przyjmij `n_init = 1` (pojedyncze uruchomienie algorytmu).

Rozwiązanie zadania:

```
random_states = [1,2,3,4,5]
max_iter = 26
#Rysowanie wykresów dla każdej inicjalizacji
plt.figure(figsize=(12, 8))
for i, state in enumerate(random_states):
    kmeans = KMeans(n_clusters=16, random_state=state, n_init=1, max_iter=max_iter, init='random')
    kmeans.fit(X_scaled)
    iter_inertia = []
    for j in range(1, max_iter+1):
        kmeans = KMeans(n_clusters=16, random_state=state, n_init=1, max_iter=j,
init='random').fit(X_scaled)
        iter_inertia.append(kmeans.inertia_)

    plt.plot(range(1, max_iter+1), iter_inertia, 'o-', label=f'random_state={state}')

plt.xlabel('Numer iteracji')
plt.ylabel('Wariancja wewnątrzgrupowa')
plt.title('Wartości wariancji')
plt.legend()
plt.grid(True)
plt.show()
```

Wynik:



Wniosek:

Analiza procesu zbieżności algorytmu K-means dla różnych wartości random_state (1-5) przy k=16 grupach wykazała następujące prawidłowości:
-Większość przypadków osiąga stabilizację wartości wariancji już przed 10 iteracją
-Największy spadek wariancji następuje w pierwszych 5 iteracjach
-Po 15 iteracjach zmiany są już minimalne
Wpływ inicjalizacji:
-Różne wartości random_state prowadzą do różnych ułożeń wykresów
-Końcowe wartości wariancji różnią się między sobą, co pokazuje wrażliwość na początkowy wybór centroidów.

Zadanie 10

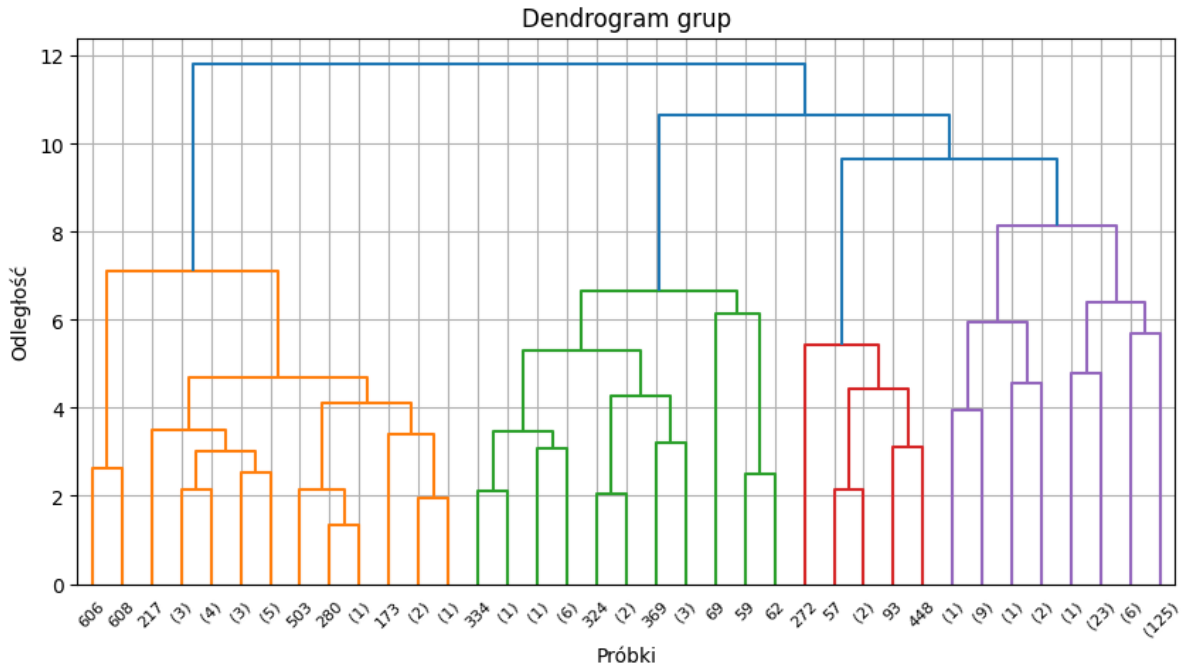
Treść:

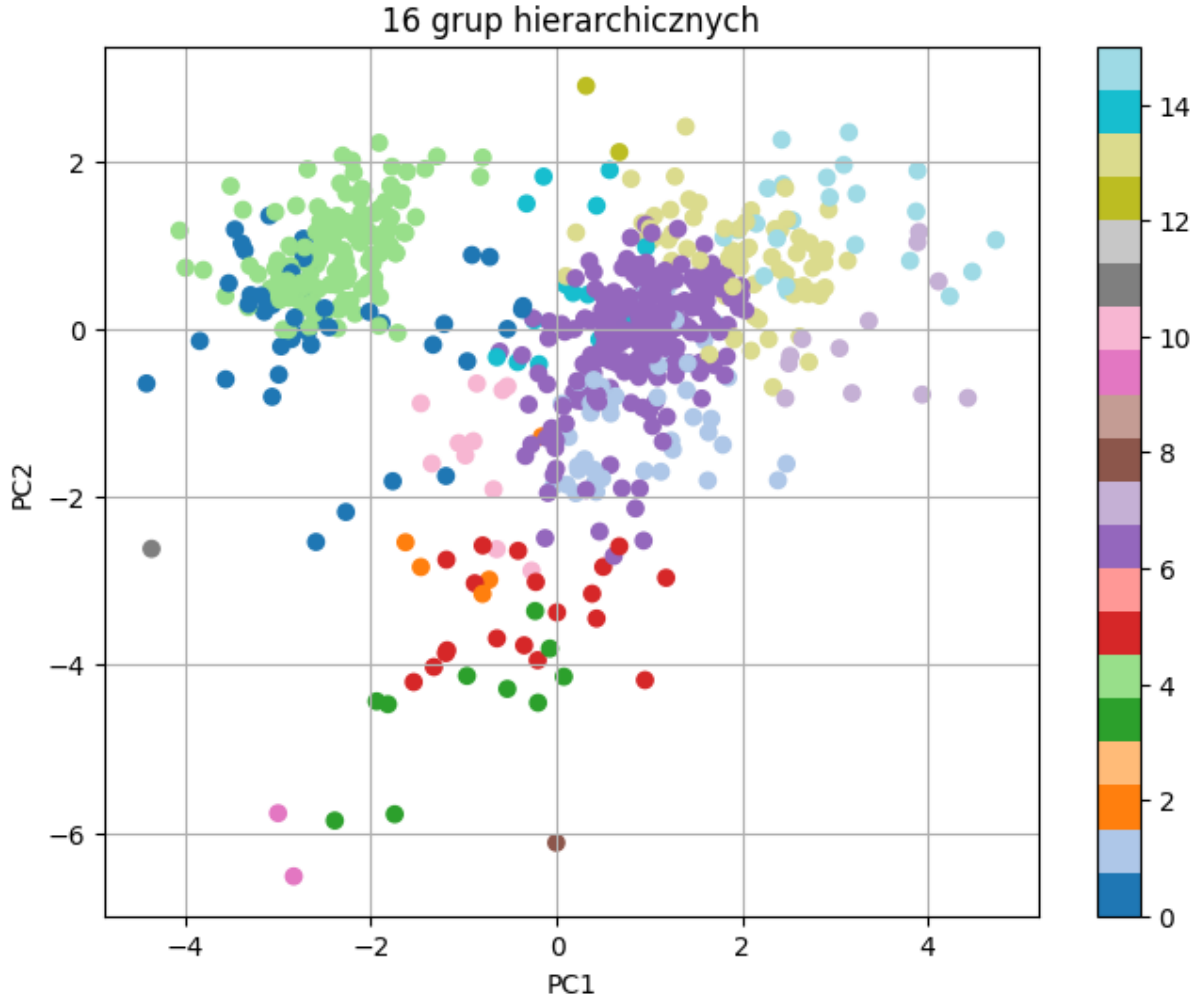
Pogrupuj profile ekspresji genów za pomocą metody hierarchicznej (patrz komórka [38] rozdz. 12.5.3).

Rozwiązanie zadania:

```
import matplotlib.pyplot as plt
from scipy.cluster.hierarchy import dendrogram, linkage
from sklearn.cluster import AgglomerativeClustering

#Dendogram
plt.figure(figsize=(10, 5))
Z = linkage(X_scaled, method='complete')
dendrogram(Z, truncate_mode='level', p=5)
plt.title('Dendrogram grup')
plt.xlabel('Próbki')
plt.ylabel('Odległość')
plt.grid(True)
plt.show()

#Grupowanie i ich wizualizacja
grupy = AgglomerativeClustering(n_clusters=16, linkage='complete').fit_predict(X_scaled)
plt.figure(figsize=(8, 6))
plt.scatter(X_pca[:, 0], X_pca[:, 1], c=grupy, cmap='tab20')
plt.title('16 grup hierarchicznych')
plt.xlabel('PC1')
plt.ylabel('PC2')
plt.colorbar()
plt.grid(True)
plt.show()
```

Wynik:


Dendrogram grup
Odległość
Próbki
0
2
4
6
8
10
12
606
608
217
(3)
(4)
(3)
(5)
503
280
(1)
173
(2)
(1)
334
(1)
(1)
(6)
324
(2)
369
(3)
69
59
62
272
57
(2)
93
448
(1)
(9)
(1)
(2)
(1)
(23)
(6)
(125)


16 grup hierarchicznych
PC1
PC2
−4
−2
0
2
4
−6
0
2
4
6
8
10
12
14

Wniosek:

W ramach zadania zaimportowano niezbędne biblioteki do analizy danych i klasteryzacji, takie W ramach hierarchicznej analizy skupień z użyciem metody complete linkage uzyskano podział genów na 16 grup. Liczebność grup jest zróżnicowana, przy czym większość koncentruje się w kilku grupach. Kilka grup zawiera pojedyncze geny, co może wskazywać na ich unikalne profile ekspresji.

Zadanie 11

Treść:

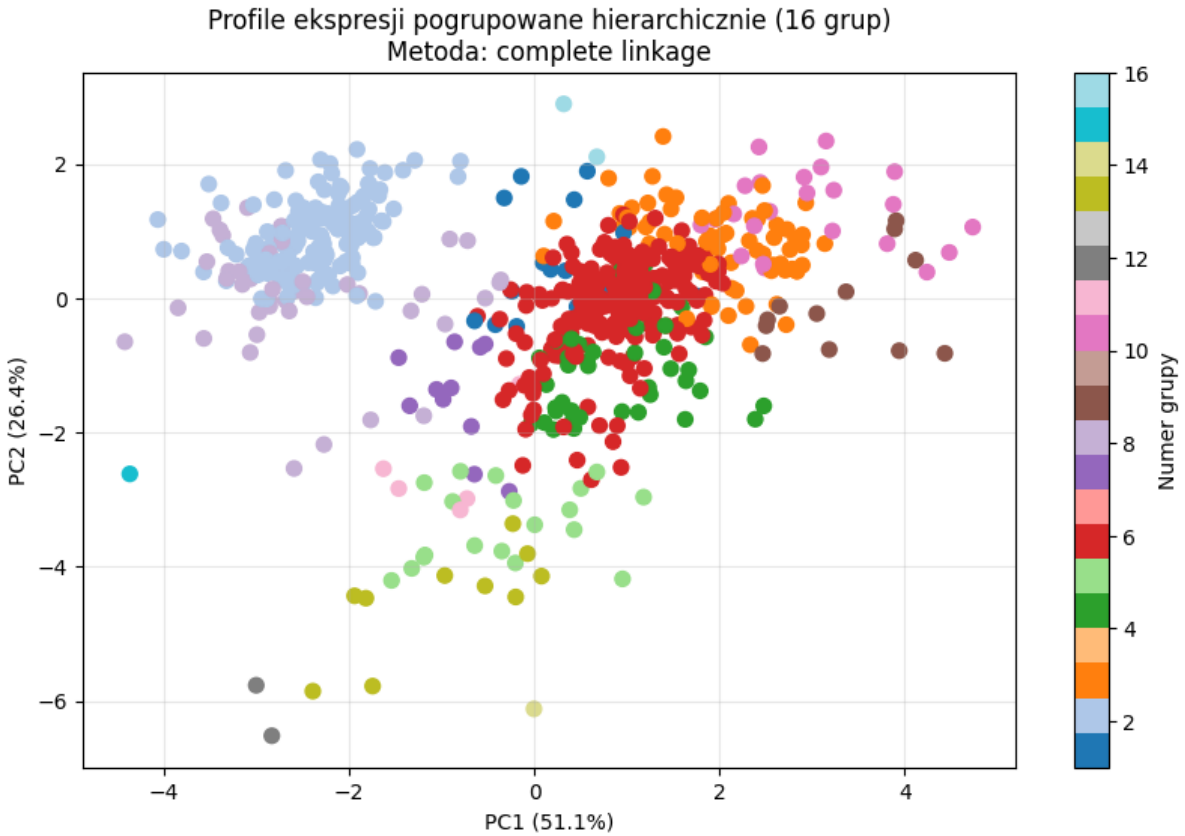
Wyznacz przynależność do grup profili ekspresji, zakładając 16 grup ([43]).

Rozwiązanie zadania:

```
from scipy.cluster.hierarchy import linkage, cut_tree
import matplotlib.pyplot as plt

#Hierarchiczne grupowanie
grupy = cut_tree(linkage(X_scaled, 'complete'), n_clusters=16).flatten() + 1
print("Liczba genów w grupach:")
for g in range(1, 17):
    print(f"Grupa {g}: {sum(grupy == g)}")

plt.figure(figsize=(10, 6))
plt.scatter(X_pca[:, 0], X_pca[:, 1], c=grupy, cmap='tab20')
plt.title('Profile ekspresji - 16 grup (complete linkage)')
plt.xlabel(f'PC1 ({pca.explained_variance_ratio_[0]*100:.1f}%)')
plt.ylabel(f'PC2 ({pca.explained_variance_ratio_[1]*100:.1f}%)')
plt.colorbar(label='Numer grupy')
plt.grid(alpha=0.3)
plt.show()
```

Wynik:

```
Liczba genów w grupach:
Grupa
1       15
2      141
3       67
4       43
5       19
6      217
7       11
8       45
9       12
10      23
11       5
12       2
13      10
14       1
15       1
16       2
```



Wniosek:

Klasteryzacja hierarchiczna z complete linkage skutecznie wyodrębniła naturalne grupy genów o podobnych profilach ekspresji. Wyniki pokazują zarówno główne wzorce ekspresji (duże grupy), jak i potencjalnie unikalne profile (małe grupy).

Zadanie 12

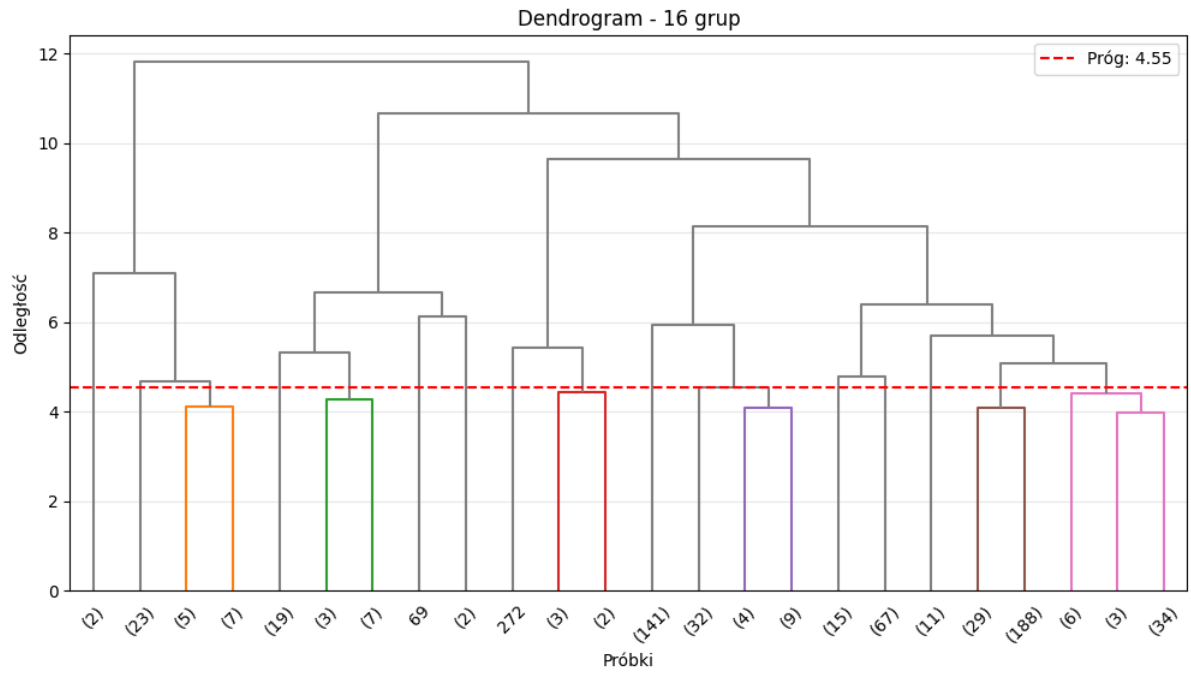
Treść:

Zobrazuj wynik dendrogramem (patrz komórki [41] i [42] rozdz. 12.5.3). Ustaw odpowiedni poziom odcięcia (`color_threshold`), zapewniający podział na 16 grup.

Rozwiązanie zadania:

```
from scipy.cluster.hierarchy import dendrogram, linkage, cut_tree
import matplotlib.pyplot as plt

#Obliczenie macierzy połączeń i progu dla 16 grup
Z = linkage(X_scaled, 'complete')
color_threshold = sorted(Z[:,2], reverse=True)[15]

#Dendogram
plt.figure(figsize=(12, 6))
dendrogram(Z, color_threshold=color_threshold,
        above_threshold_color='gray',
        truncate_mode='lastp', p=24)

plt.title('Dendrogram - 16 grup')
plt.xlabel('Próbki')
plt.ylabel('Odległość')
plt.axhline(color_threshold, color='r', linestyle='--', label=f'Próg: {color_threshold:.2f}')
plt.legend()
plt.grid(axis='y', alpha=0.3)
plt.show()

#Liczebności grup
print(f"Liczba utworzonych grup: {len(set(cut_tree(Z, n_clusters=16).flatten()))}")
```

Wynik:




```
Liczba utworzonych grup: 16
```

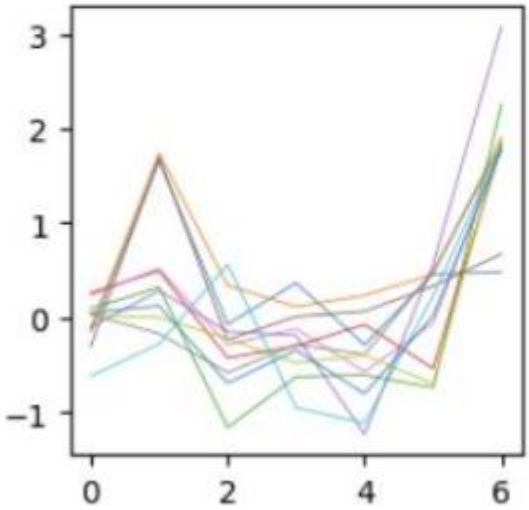
Wniosek:

Próg odcięcia wynosi 4,55. Oznacza to, że na poziomie 4,55 powstanie 16 oddzielnych grup.

Zadanie 13

Treść:

Zwizualizuj profile należące do poszczególnych grup.
16 wykresów postaci:



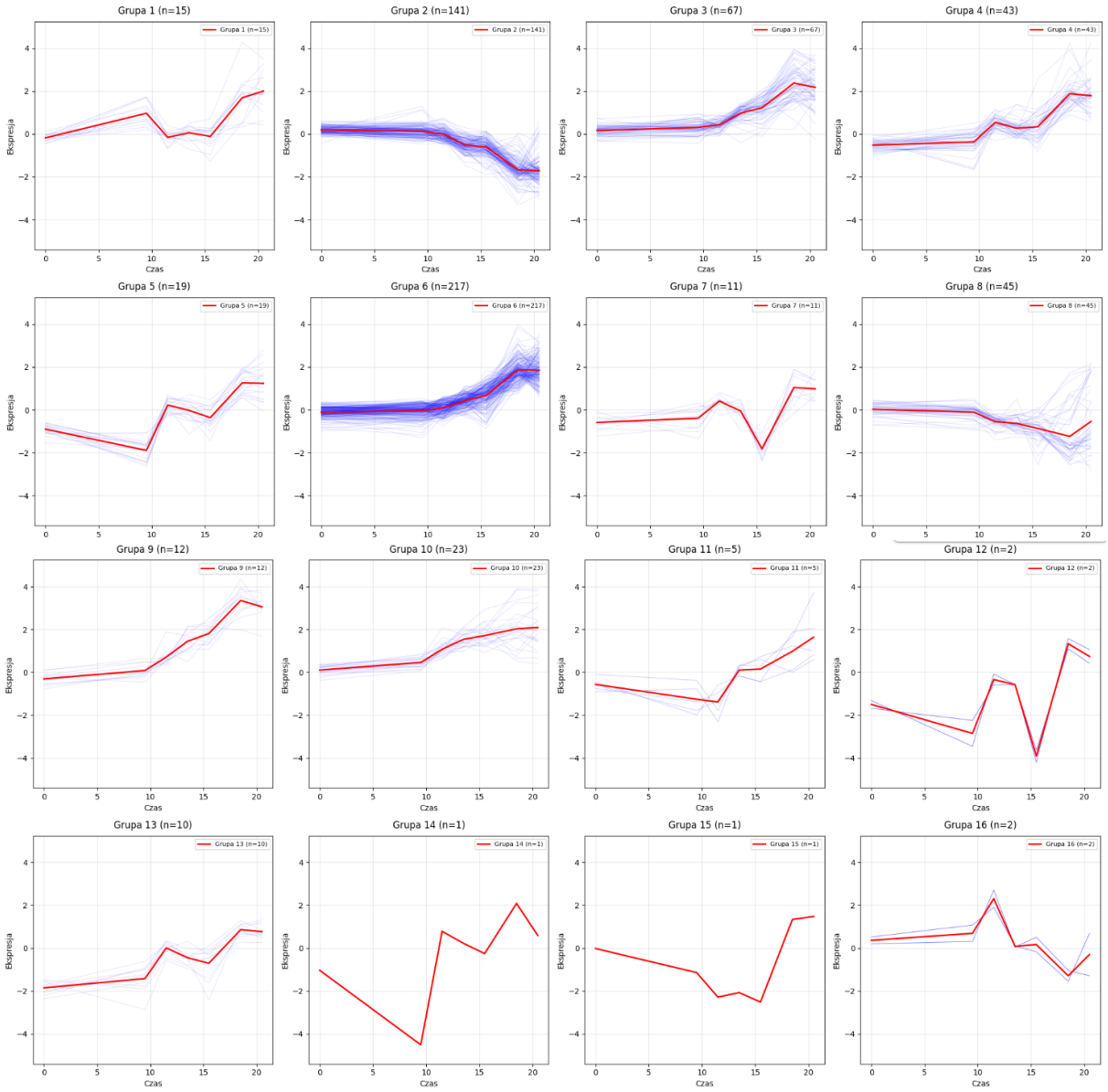
Rozwiązanie zadania:

```
import matplotlib.pyplot as plt
import numpy as np

grupy = cut_tree(linkage(X_scaled, 'complete'), n_clusters=16).flatten()
#Tworzenie wykresów
plt.figure(figsize=(15, 15))
for i in range(16):
    plt.subplot(4, 4, i+1)

    #Profile genów
    maska = (grupy == i)
    for profil in ekspresja[maska]:
        plt.plot(czas, profil, 'b-', alpha=0.2)

    #Centroidy
    centroid = scaler.inverse_transform(X_scaled[maska].mean(axis=0).reshape(1, -1))[0]
    plt.plot(czas, centroid, 'r-', linewidth=2)


    plt.title(f'Grupa {i+1} (n={sum(maska)})')
    plt.ylim(ekspresja.min(), ekspresja.max())
    plt.grid(alpha=0.3)

plt.tight_layout()
plt.show()
```

Wynik:

Liczebność grup przed filtracją

0	15
1	141
2	67
3	43
4	19
5	217
6	11
7	45
8	12
9	23
10	5
11	2
12	10
13	1
14	1
15	2



Wniosek:

Grupa 6 i Grupa 2 są najliczniejsze, co sugeruje, że zawierają geny o podobnych wzorcach ekspresji, które są powszechne w danych. Z kolei grupy 11–16 (1–5 genów) to geny o unikalnych profilach ekspresji.
Silna dysproporcja w liczebnościach między grupami, metoda hierarchiczna lepiej wychwytuje naturalną strukturę danych niż k-średnich która zwykle daje bardziej zbalansowane grupy.

Zadanie 14

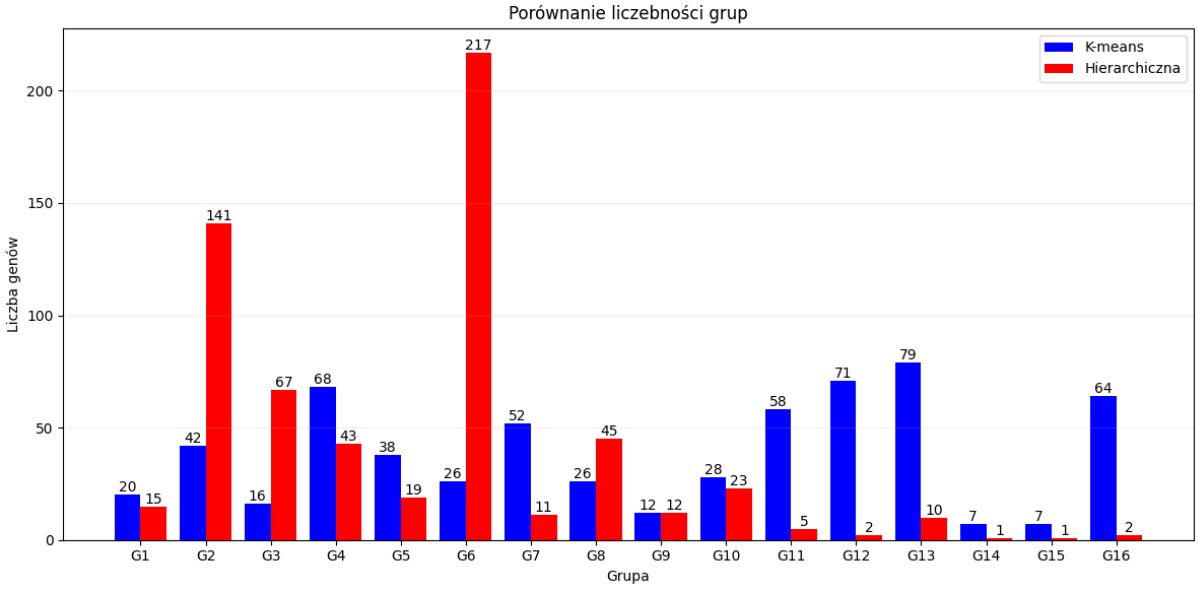
Treść:

Zobrazuj na wykresie słupkowych liczebności grup dla metody k-means i aglomeracyjnej.

Rozwiązanie zadania:

```
import matplotlib.pyplot as plt
import numpy as np

kmeans = KMeans(n_clusters=16, random_state=42).fit(X_scaled)
hier = cut_tree(linkage(X_scaled, 'complete'), n_clusters=16).flatten()

#Liczebności dla obu metod w każdej grupie
kmeans_counts = [sum(kmeans.labels_ == i) for i in range(16)]
hier_counts = [sum(hier == i) for i in range(16)]
#Wykres
plt.figure(figsize=(12, 6))
x = np.arange(16)
width = 0.4
plt.bar(x - width/2, kmeans_counts, width, label='K-means', color='blue')
plt.bar(x + width/2, hier_counts, width, label='Hierarchiczna', color='red')
plt.xticks(x, [f'G{i+1}' for i in range(16)])
plt.xlabel('Grupa')
plt.ylabel('Liczba genów')
plt.title('Porównanie liczebności grup')
plt.legend()
plt.grid(axis='y', alpha=0.2)
#Etykiety dla danych
for i in x:
    plt.text(i - width/2, kmeans_counts[i], str(kmeans_counts[i]), ha='center', va='bottom')
    plt.text(i + width/2, hier_counts[i], str(hier_counts[i]), ha='center', va='bottom')

plt.tight_layout()
plt.show()
```

Wynik:



Wniosek:

W metodzie hierarchicznej powstała jedna bardzo liczba grupa (6). K-means ma bardziej rozłożony podział pomiędzy grupami.

Zadanie 15

Treść:

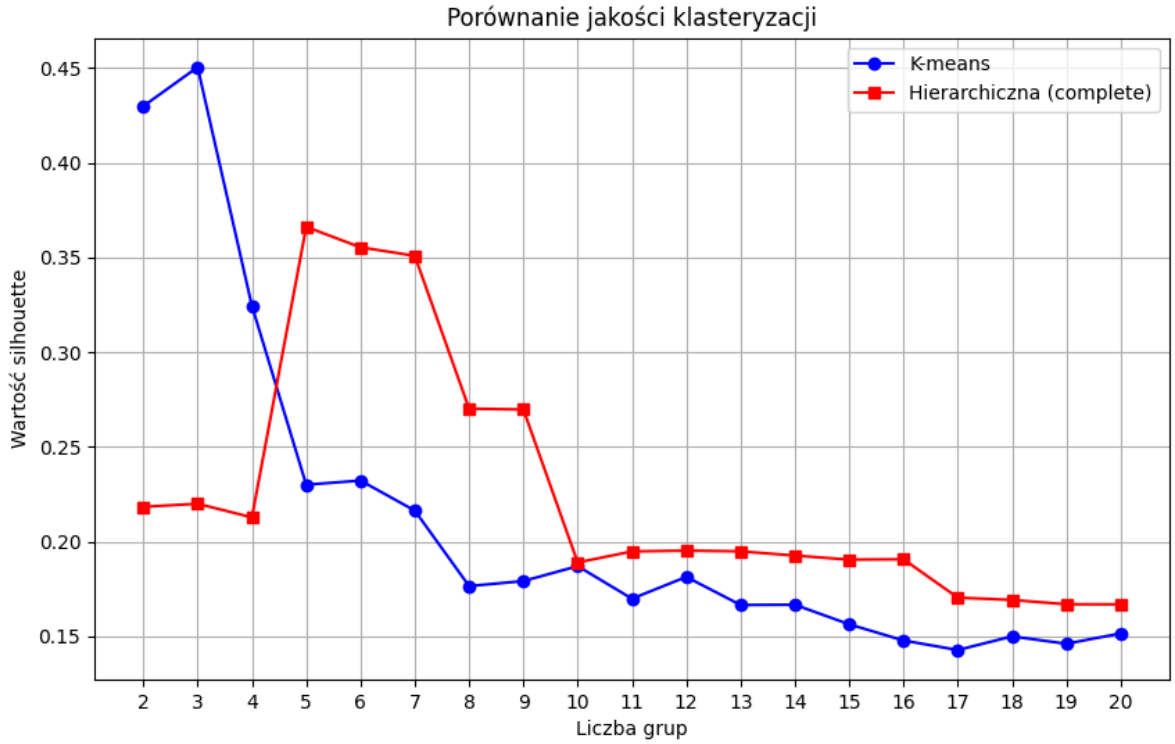
Zobrazuj na wykresie wartości metryki silhouette (`silhouette_score`) dla obu algorytmów i różnej liczby grup.

Rozwiązanie zadania:

```
from sklearn.metrics import silhouette_score

cluster_range = range(2, 21)
silhouette_kmeans = []
silhouette_hier = []

for n in cluster_range:
  #Sylwetka dla k-średnich
    kmeans = KMeans(n_clusters=n, random_state=42).fit(X_scaled)
    silhouette_kmeans.append(silhouette_score(X_scaled, kmeans.labels_))
  #Sylwetka dla aglomeracyjnej
    hier = AgglomerativeClustering(n_clusters=n, linkage='complete').fit(X_scaled)
    silhouette_hier.append(silhouette_score(X_scaled, hier.labels_))
```

```
plt.figure(figsize=(10, 6))
plt.plot(cluster_range, silhouette_kmeans, 'bo-', label='K-means')
plt.plot(cluster_range, silhouette_hier, 'rs-', label='Hierarchiczna (complete)')
plt.xlabel('Liczba grup')
plt.ylabel('Wartość silhouette')
plt.title('Porównanie jakości klasteryzacji')
plt.xticks(cluster_range)
plt.grid()
plt.legend()
plt.show()
```

Wynik:



Wniosek:

Dla metody K-średnich najwyższą wartość silhouette (najlepszą jakość grupowania) uzyskano dla k=3, a dla metody hierarchicznej (complete linkage) najlepszy wynik osiągnięto dla k=5. K-średnich dało lepsze wyniki (wyższe wartości silhouette) dla małych wartości k (2-5)
Metoda hierarchiczna wypadła lepiej dla większych k , mimo, że nie są to zbyt satysfakcjonujące wyniki. Im bliżej 1 tym lepiej.

Zadanie 16

Treść:

. Do której grupy utworzonej przez algorytm k-średnich trafi profil x = [-0.2205, -0.0041, 0.3821, 0.3680, 0.4918, 1.6983, 1.9820]? Narysuj ten profil na tle wszystkich profili z grupy, do której został przypisany. Wypisz nazwy wszystkich genów z tej grupy.

Rozwiązanie zadania:

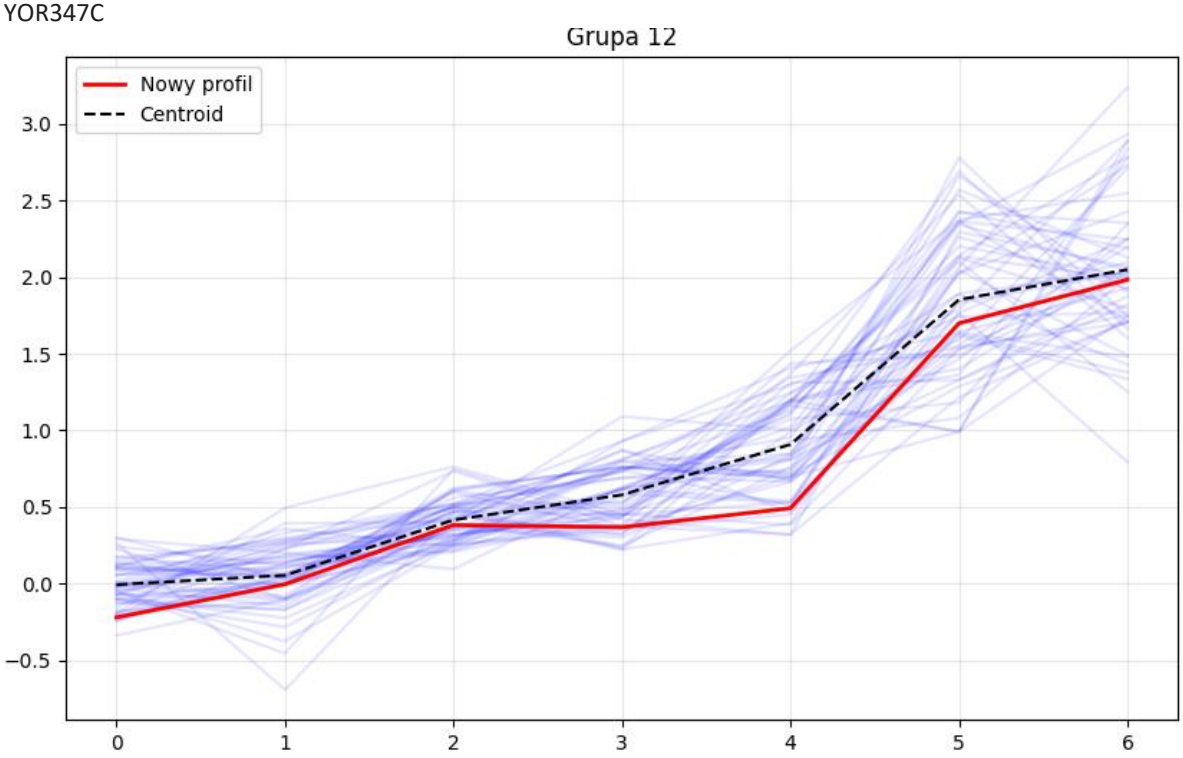
```
profil_skalowany = scaler.transform([[-0.2205, -0.0041, 0.3821, 0.3680, 0.4918, 1.6983, 1.9820]]) #Nowy
punkt
grupa = KMeans(n_clusters=16, random_state=2).fit(X_scaled).predict(profil_skalowany)[0] + 1
#Fitowanie punktu do odpowiedniej grupy
print(f"Profil należy do grupy: {grupa}")

#Wypisywanie genów
geny_grupy = [geny[i] for i, label in enumerate(kmeans.labels_) if label == grupa-1]
print(f"\nGeny w grupie {grupa} ({len(geny_grupy)}):\n" + "\n".join(geny_grupy))

plt.figure(figsize=(10, 6))

#Profile
for i in np.where(kmeans.labels_ == grupa-1)[0]:
    plt.plot(ekspresja[i], 'b-', alpha=0.1)

#Przynależność do profilu
plt.plot(nowy_profil, 'r-', linewidth=2, label='Nowy profil')
plt.plot(scaler.inverse_transform([kmeans.cluster_centers_[grupa-1]])[0], 'k--', label='Centroid')

plt.title(f'Grupa {grupa}')
plt.legend()
plt.grid(alpha=0.3)
plt.show()
```

Wynik:

Profil należy do grupy: 12

Geny w grupie 12 (53):
YDR272W
YGR111W
YGR201C
YLR168C
YLL023C
YMR181C
YOR220W
YBR214W
YDL181W
YDR272W
YGL259W
YHR051W
YKR058W
YKR076W
YOL053W
YOL071W
YOR273C
YOR289W
YPL165C
NORF 4
YBL107C
YEL060C
YGL192W
YGL198W
YGR149W
YKL148C
YMR090W
YMR110C
YMR191W
YNL305C
YPL078C
YBL100C
YBL108W
YBR039W
YBR230C
YCL025C
YDR059C
YDR277C
YDR513W
YGR028W
YGR194C
YHL024W
YKR016W
YLR395C
YMR031C
YMR056C
YMR188C
YMR197C
YOL153C
YOL083W
YOR065W

YOR285W
YOR347C



Wniosek:

Punkt został przypisany do grupy 12, wynik przypisania jest zależny od random state, ponieważ wybierany jest w ten sposób inny centroid początkowy mający duży wpływ na wynik przynależności do grupy. Inny random state zmieni jego przynależność. Wykres genu jest zbliżony do wyglądu centroidu.

Wnioski końcowe:

Przeprowadzone ćwiczenie miało na celu zastosowanie metod klasteryzacji do analizy danych genetycznych, w szczególności identyfikację grup genów o podobnych profilach ekspresji w czasie. Wykorzystano dwie główne metody: K-średnich oraz klasteryzację hierarchiczną z łączem kompletnym (complete linkage), uzupełnione o analizę głównych składowych (PCA) dla redukcji wymiarowości danych. Klasteryzacja K-średnich dla k=16 grup wykazała bardziej zrównoważony rozkład genów między klastrami, podczas gdy metoda hierarchiczna wyodrębniła jedną dominującą grupę oraz kilka mniejszych, co sugeruje istnienie wyraźnego "rdzenia" podobnych genów. Analiza metody łokcia i współczynnika silhouette wskazała, że optymalna liczba grup to 3-5, co potwierdza, że wybór k=16 mógł prowadzić do nadmiernego podziału. Wizualizacja w przestrzeni PCA (PC1 vs PC2) ujawniła dobre rozdzielenie głównych grup, choć niektóre obszary nakładały się, co może wynikać z podobieństw w profilach ekspresji. Przypisanie nowego profilu ekspresji do grupy nr 12 wykazało praktyczne zastosowanie metody, potwierdzając jej przydatność w klasyfikacji. Podsumowując, ćwiczenie potwierdziło, że wybór metody klasteryzacji znacząco wpływa na wyniki, a metoda hierarchiczna lepiej oddaje naturalną strukturę danych, podczas gdy K-średnich zapewnia bardziej równomierny podział.